



Data cleaning 2

Kristoffer Wild

Lectures: Monday 9 to 10:30am PAR 166-L1-103 (Old Metallurgy) - Once a week

Workshops: Wednesday 11 to 1pm apar 125-L1-127 WEBS - Once a week (weeks 2-11)

Teaching week	Date	Who will be teaching	Lecture topic	Workshop	Quizzes	Weekly reading (included in quiz)
Week 1	28-Jul	Iliana Medina	Welcome, introduction to subject and R selfassessment	NO WORKSHOP		NO READING
Week 2	4-Aug	Kristoffer Wild	Introduction to R, creating projects and learning to use AI	Description of semester projects, choice of project groups, creation of R project	Quiz 1	NO READING
Week 3	11-Aug	Kristoffer Wild	Data cleaning and exploration 1	Start working with project dataset, understanding data, cleaning dataset		Zuur, A. F., Ieno, E. N., & Elphick, C. S. (2010). A protocol for data exploration to avoid common statistical problems. <i>Methods in ecology and evolution</i> , 1 (1), 3-14.
Week 4	18-Aug	Kristoffer Wild	Data cleaning and exploration 2	Exploration of project datasets	Quiz 2	NO READING
Week 5	25-Aug	Liz Bromfield	Transparency and reproducibility	Research integrity and public perceptions of science	Coding assignment (20%) due on Friday 29th.	<u>Baker, M. (2016). 1500 scientists lift the lid on reproducibility. <i>Nature</i>. 533. 452-454.</u>

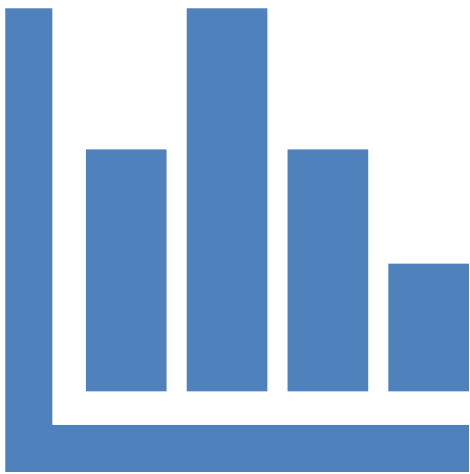
Today's learning objectives



- Group problem and solution (25min)
- Transforming your data (5min)
- New functions for data cleaning (30min)
 - pivots, binds, joins, gsub, if_else
- Apply with your group data (15min)
- Review content for upcoming quiz/questions (15 min)

- Break out in your groups (4min)
 - identify your problem and your solution
 - identify a problem you haven't resolved

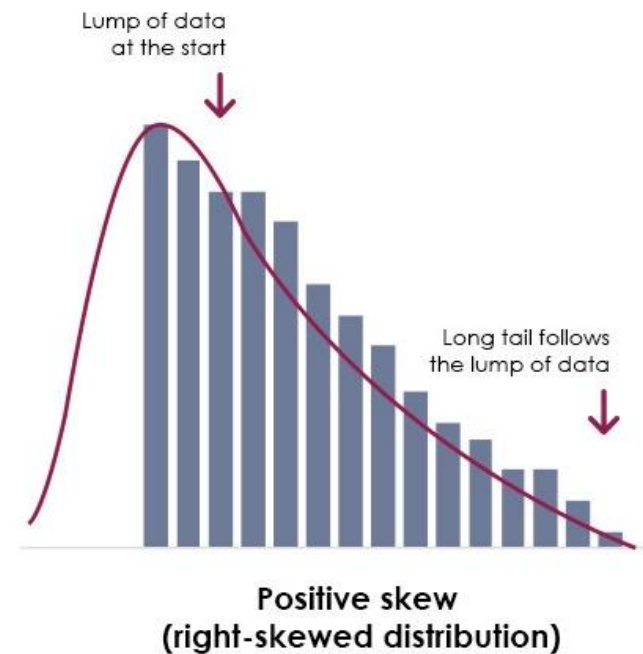
Be ready to share to the class!



TRANSFORMING
YOUR DATA

Transformations for Right (Positive) Skew

- Use these transformations in increasing order of strength:
- Root ($\sqrt{x}$, $\sqrt[3]{x}$): Mild transformation for slight skew.
- Logarithm ($\log(x)$): Stronger; shift data to avoid $\log(0)$ using $\log(x + |\min(x)| + 1)$.
- Reciprocal ($1/x$, $1/x^3$): Strongest; avoid near-zero values by shifting.



R Code: Transformations for Right-Skewed Data

```
# Example: Right-Skewed Data; simulate the data
set.seed(42)
x <- rexp(1000, rate = 0.2)

# Root transformations
sqrt_x <- sqrt(x)
cube_root_x <- x^(1/3)

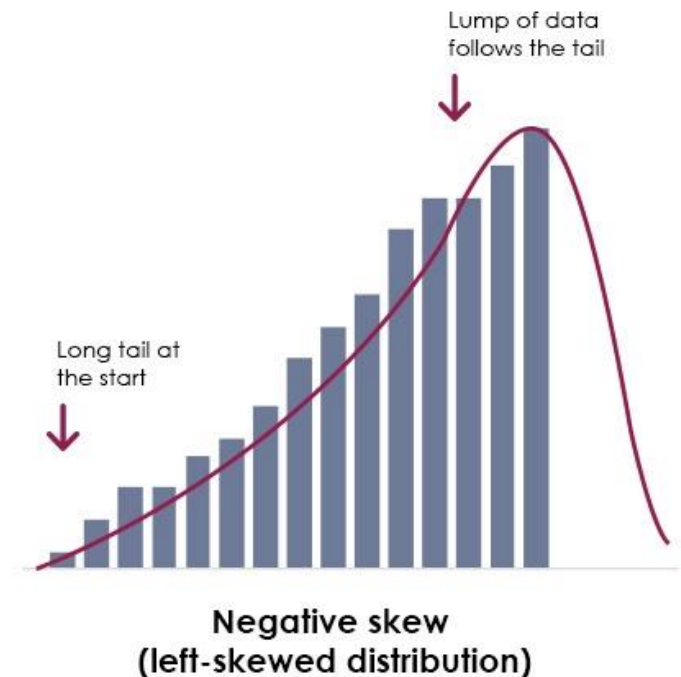
# Log transformations
log_x <- log(x)
log10_x <- log10(x)
log1p_x <- log1p(x) # safer for zeros

# Shifted log to handle 0 or negatives
x_shifted <- x - min(x) + 1
log_shifted <- log(x_shifted)

# Reciprocal transformations
recip_x <- 1 / x
recip_x3 <- 1 / (x^3)
```

Transformations for Left (Negative) Skew

- Reflect the data first, then apply transformations:
- Reflect using: $\max(x) + 1 - x$ to reverse the skew direction.
- Square (x^2): Mild to moderate transformation; not suitable for negative values.
- Exponential (e^x): Strongest transformation; handles negative input well.



R Code: Transformations for Left-Skewed Data

```
# Example: Left-Skewed Data simulate the data
```

```
set.seed(42)
```

```
x_left <- -rexp(1000, rate = 0.2)
```

```
# Reflect the data
```

```
x_reflected <- max(x) + 1 - x
```

```
# Apply transformations to reflected data
```

```
square_x <- x_reflected^2
```

```
exp_x <- exp(x_reflected)
```

```
log_reflected <- log(x_reflected)
```

```
log10_reflected <- log10(x_reflected)
```

PIVOTING YOUR DATA



pivot_longer()

country	1999	2000	2001	2002
Angola	800	750	925	1020
India	20100	25650	26800	27255
Mongolia	450	512	510	586

Pivot data longer

```
data %>%  
  pivot_longer(  
    cols = 1999:2002,  
    names_to = "year",  
    values_to = "cases"  
  )
```



country	year	cases
Angola	1999	800
Angola	2000	750
Angola	2001	925
Angola	2002	1020
India	1999	20100
India	2000	25650
India	2001	26800
India	2002	27255
Mongolia	1999	450
Mongolia	2000	512
Mongolia	2001	510
Mongolia	2002	586

- Converts wide data into long format.
- Turns column names into values under a new variable.
- Used for tidying data before visualization or analysis.
- Syntax: `pivot_longer(data, cols, names_to, values_to)`

pivot_longer() in Action

```
```r
library(tidyr)
data <- data.frame(ID = c(1, 2), Jan = c(10, 20), Feb = c(15, 25))

pivot_longer(data, cols = Jan:Feb, names_to = 'Month', values_to = 'Value')
```

## # Output:

```
A tibble: 4 × 3
 ID Month Value
 <dbl> <chr> <dbl>
1 1 Jan 10
2 1 Feb 15
3 2 Jan 20
4 2 Feb 25
```

# pivot\_wider()

country	year	cases
Angola	1999	800
Angola	2000	750
Angola	2001	925
Angola	2002	1020
India	1999	20100
India	2000	25650
India	2001	26800
India	2002	27255
Mongolia	1999	450
Mongolia	2000	512
Mongolia	2001	510
Mongolia	2002	586

country	1999	2000	2001	2002
Angola	800	750	925	1020
India	20100	25650	26800	27255
Mongolia	450	512	510	586

**Pivot data wider**

```
data %>%
 pivot_wider(
 names_from = "year",
 values_from = "cases"
)
```

- Converts long data into wide format.
- Spreads values across multiple columns using a key column.
- Useful when summarizing tidy data.
- Syntax: `pivot_wider(data, names_from, values_from)`

# pivot\_wider() in Action

```
```r
library(tidyr)
data <- data.frame(
  ID = c(1, 1, 2, 2),
  Month = c('Jan', 'Feb', 'Jan', 'Feb'),
  Value = c(10, 15, 20, 25))

pivot_wider(data, names_from = Month, values_from = Value)
```

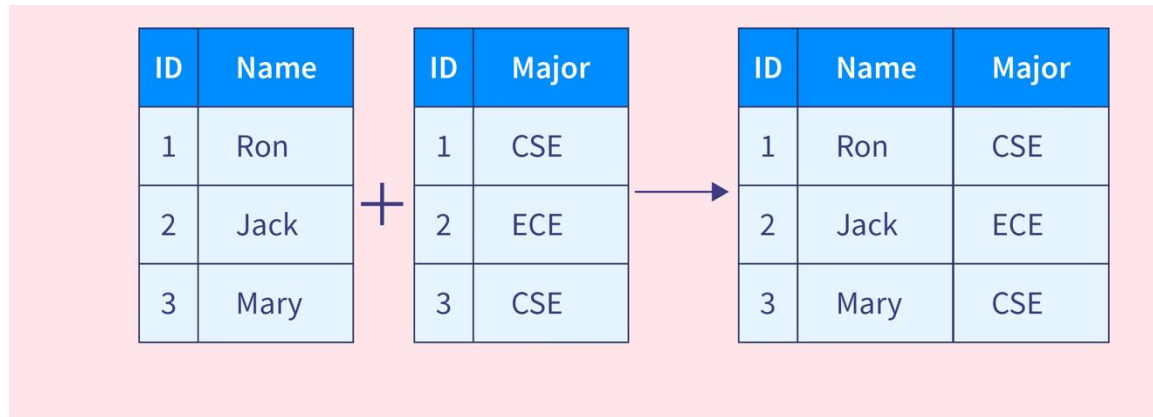
Output:

```
# A tibble: 2 × 3
  ID   Jan   Feb
1   1    10    15
2   2    20    25
```



BINDING YOUR DATA

cbind()



- Combines dataframes ****by columns****, binding them side-by-side.
- Used when data has the ****same number of rows**** and you want to add variables.
- Base R function (no packages needed).
- Syntax: `cbind(df1, df2)`

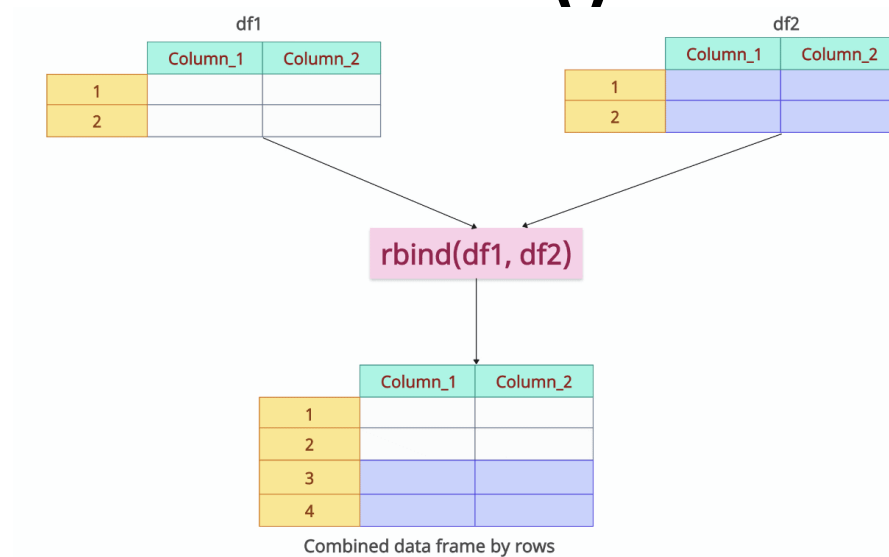
cbind() in Action

```
```r  
df1 <- data.frame(ID = 1:3)
df2 <- data.frame(Name = c('A', 'B', 'C'))
cbind(df1, df2)
```

## # Output:

	ID	Name
1	1	A
2	2	B
3	3	C

# rbind()



- Combines dataframes **\*\*by rows\*\***, stacking them vertically.
- Used when data has the **\*\*same columns\*\*** and you want to add observations.
- Base R function (no packages needed).
- Syntax: `rbind(df1, df2)`

# rbind() in Action

```
` ``r
df1 <- data.frame(ID = 1:2, Name = c('A', 'B'))
df2 <- data.frame(ID = 3, Name = 'C')
rbind(df1, df2)
```

## # Output:

	ID	Name
1	1	A
2	2	B
3	3	C



Joining your data

# inner\_join()

ID	X1	ID	X2
1	a1	2	b1
2	a2	3	b2

inner\_join

ID	X1	X2
2	a2	b1

- Combines two dataframes by common key.
- Returns only rows with matching keys in both dataframes.
- Rows without a match are excluded.
- Syntax: `inner_join(df1, df2, by = 'key')`

# inner\_join() in Action

```
```r
library(dplyr)
df1 <- data.frame(ID = c(1, 2, 3), Name = c('A', 'B', 'C'))
df2 <- data.frame(ID = c(2, 3, 4), Score = c(80, 90, 70))
inner_join(df1, df2, by = 'ID')
```

Output:

	ID	Name	Score
1	2	B	80
2	3	C	90

ID	X1
1	a1
2	a2

ID	X2
2	b1
3	b2

inner_join

ID	X1	X2
2	a2	b1

left_join()

ID	X1
1	a1
2	a2

ID	X2
2	b1
3	b2

left_join

ID	X1	X2
1	a1	NA
2	a2	b1

- Combines two dataframes by key.
- Returns all rows from the left dataframe.
- Adds matching rows from right; unmatched = NA.
- Syntax: `left_join(df1, df2, by = 'key')`

left_join() in Action

```
```r  
left_join(df1, df2, by = 'ID')
```

## # Output:

	ID	Name	Score
1	1	A	NA
2	2	B	80
3	3	C	90

ID	X1
1	a1
2	a2

ID	X2
2	b1
3	b2

left\_join

ID	X1	X2
1	a1	NA
2	a2	b1

# right\_join()

ID	X1	ID	X2
1	a1	2	b1
2	a2	3	b2

right\_join

ID	X1	X2
2	a2	b1
3	NA	b2

- Combines two dataframes by key.
- Returns all rows from the right dataframe.
- Adds matching rows from left; unmatched = NA.
- Syntax: `right_join(df1, df2, by = 'key')`

# right\_join() in Action

```
```r  
right_join(df1, df2, by = 'ID')
```

Output:

	ID	Name	Score
1	2	B	80
2	3	C	90
3	4	<NA>	70

ID	X1	ID	X2
1	a1	2	b1
2	a2	3	b2

right_join

ID	X1	X2
2	a2	b1
3	NA	b2

full_join()

ID	X1
1	a1
2	a2

ID	X2
2	b1
3	b2

full_join

ID	X1	X2
1	a1	NA
2	a2	b1
3	NA	b2

- Combines two dataframes by key.
- Returns all rows from both dataframes.
- Unmatched rows get NA values for missing columns.
- Syntax: `full_join(df1, df2, by = 'key')`

full_join() in Action

```
```r  
full_join(df1, df2, by = 'ID')
```

## # Output:

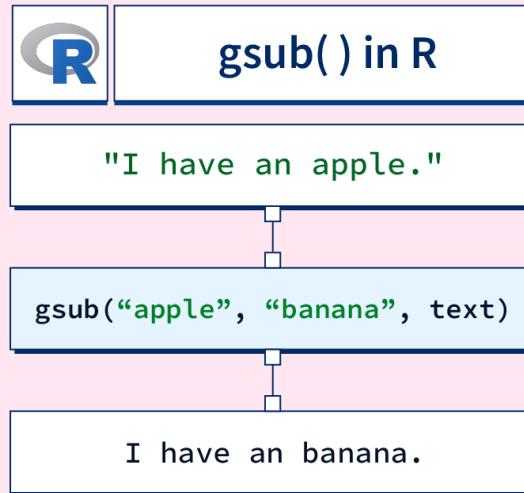
	ID	Name	Score
1	1	A	NA
2	2	B	80
3	3	C	90
4	4	<NA>	70

ID	X1	ID	X2
1	a1	2	b1
2	a2	3	b2

full\_join

ID	X1	X2
1	a1	NA
2	a2	b1
3	NA	b2

# gsub()



- Replaces all matches of a pattern in a string.
- Commonly used for text cleaning.
- Syntax: `gsub('pattern', 'replacement', x)`
- Returns a character vector with substitutions.

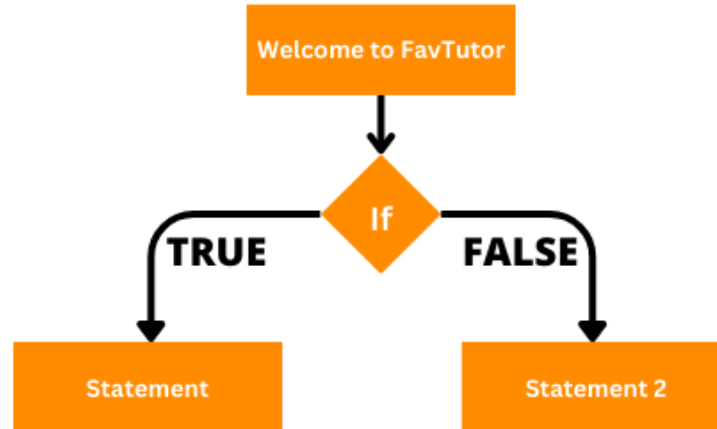
# gsub() in Action

```
```r  
text <- c('apples', 'bananas', 'apples')  
gsub('apples', 'oranges', text)
```

Output:

```
[1] 'oranges' 'bananas' 'oranges'
```

if_else()



- Vectorized conditional statement from dplyr.
- Returns value A if TRUE, value B if FALSE.
- Safer and faster than base `ifelse()`.
- Syntax: `if_else(condition, true, false)`

if_else() in Action

```
` `` `r  
x <- c(10, 5, 8)  
if_else(x > 7, 'high', 'low')
```

Output:

```
[1] 'high' 'low' 'high'
```



PRACTICE WITH YOUR DATA

QUIZ 2

- Go over dplyr verbs
 - use the [dplyr cheat sheet](#)
- Practice connecting the verbs with %>%
- READ over Zuur paper and know the protocol for cleaning data
- Practice pivoting and joining datasets!